```
#%%
from datetime import datetime, timedelta
from statsmodels.tsa.seasonal import
seasonal_decompose
import numpy as np
import pandas as pd

from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf,
plot_pacf
from statsmodels.stats.diagnostic import
acorr_ljungbox
from sklearn.metrics import mean_absolute_error,
root_mean_squared_error
import itertools

import warnings

from tensorflow.core.distributed_runtime import
preemption
from statsmodels.tsa.api import ExponentialSmoothing,
SimpleExpSmoothing,Holt

warnings.filterwarnings("ignore")
#%% md
#### Importing the dataset comprising data of
children with illegal immigration status or no parent
 or guardian
#%%
df = pd.read_csv("
HHS_Unaccompanied_Alien_Children_Program.csv")
df.head()
#%% md
#### Checking for null values and duplicates
#%%
df.isna().sum()
#%%
df.duplicated().sum()
#%% md
#### Converting data to pandas date-time format
```

```
#%%
df["Date"] = pd.to_datetime(df["Date"])
df.head()
```

```
#%% md
#### Arrange dates in ascending order
```

```
#%%
df.sort_values(by=["Date"], inplace=True)
df.reset_index(drop=True, inplace=True)
df.head()
```

```
#%% md
#### Retrieving weekday names for each date in the dataset
```

```
#%%
df["Day of Week"] = list(map(lambda x: x.strftime("%A"), df["Date"]))
df = df[["Date","Day of Week","Children apprehended and placed in CBP custody*","Children in CBP custody","Children transferred out of CBP custody","Children in HHS Care","Children discharged from HHS Care"]]
df.head()
```

```
#%% md
#### Finding out the total number of records on each day of week
```

```
#%%
weekday_number = df["Day of Week"].value_counts().reset_index()
weekday_number["Day of Week Number"] = weekday_number["Day of Week"].map({
    "Sunday": "1",
    "Monday": "2",
    "Tuesday": "3",
    "Wednesday": "4",
    "Thursday": "5",
    "Friday": "6",
    "Saturday": "7"
})
weekday_number = weekday_number.sort_values(by=["Day of Week Number"]).reset_index(drop=True)
weekday_number
```

```
#%% md
Looking at the numbers above, we can see that the
```

```
number of days on which children were recorded is relatively consistent from Sunday through Thursday. From Monday through Thursday, the figures range between 145 and 150 days. Sunday is somewhat lower, at 130 days, but it is still reasonably close to this range.

Friday, however, stands out as an exceptionally low number of recorded days. Data is recorded for only two Fridays, compared with more than 130 recorded days for each of the other weekdays. This substantial difference suggests that Friday may represent a distinct pattern in the data and warrants further investigation.

There is one assumption that we can make here, which is deleting those two Friday records and assuming that services are not provided on Fridays and Saturdays. But in reality, these services are provided 24 hours a day and 7 days a week. So we need to include these days on which data is not recorded and then somehow deal with the null values for number of children on these days.
```

```
#%% md
#### Creating a new dataset with dates on which data is not recorded
```

```
#%%
df_2 = pd.DataFrame(columns=df.columns)
df_2["Date"] = np.setdiff1d(pd.date_range(df["Date"].min(),df["Date"].max()).values,df["Date"].values)
df_2["Day of Week"] = list(map(lambda x: x.strftime("%A"), df_2["Date"]))
df_2[['Children apprehended and placed in CBP custody*','Children in CBP custody','Children transferred out of CBP custody','Children in HHS Care', 'Children discharged from HHS Care']] = np.nan
df_2.head()
```

```
#%% md
#### Concatenating the original dataset with the new dataset comprising unrecorded dates
```

```
#%%
df_3 = pd.concat([df,df_2]).sort_values(by=["Date"]).reset_index(drop=True)
df_3.drop(columns=["Day of Week"], inplace=True)
df_3.set_index("Date", inplace=True)
df_3.head()
#%% md
#### Checking the data types of each feature
#%%
df_3.dtypes
#%% md
As we observe the above data types, the feature *Children in HHS care* is of object type. The other features are of float type. In reality, these features denote number of children going through their respective stage. This means that such features ought to be integers.

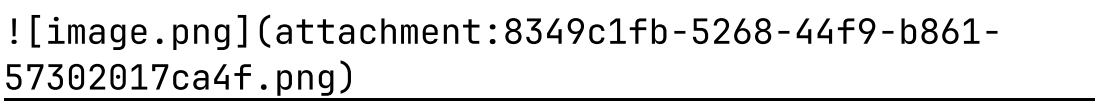
All the features except the *Children in HHS Care* are floating points, which makes it unnecessary for us to convert into integers, as they are of numeric type. The values for such features end with ".0". The *Children in HHS Care* is of object type, which is a non-numeric type - this forces us to convert it to numeric type. Before type conversion, let us see how the values of *Children in HHS Care* look like:
#%%
print(df_3["Children in HHS Care"].unique().tolist())
#%% md
As we observe, there are commas in each element of *Children in HHS Care* column. We need to remove those commas and convert the data type into floating point so that the data type matches with that of other columns.
#%%
df_4 = df_3.copy()

df_4["Children in HHS Care"] = list(map(lambda x: "".join(x.split(",")), df_4["Children in HHS Care"]))
#%% md
```

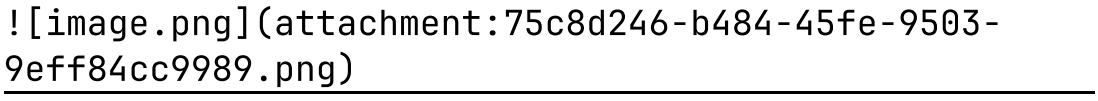
```
From the above error, we can say that there are some floating point values for the column *Children in HHS Care*. We need to explicitly convert all values to string, then remove those commas and then convert them to floating point.
#%%
df_4["Children in HHS Care"] = list(map(lambda x: "".join(str(x).split(",")),df_4["Children in HHS Care"]))
df_4["Children in HHS Care"] = df_4["Children in HHS Care"].astype("float64")
df_4.head()
#%% md
Now, let us make use of data interpolation, which figures out values for each null value by itself based on non-null values around each of them.
#%%
df_4 = df_4.interpolate().copy()
df_4.head()
#%% md
As we observe the above dataset, the values are in decimal points. In reality the features denote number of children in each phase - this means that the elements should also be integers. So, we shall round all the elements to their nearest integers.
#%%
df_5 = df_4.round(0)
df_5.head()
#%% md
Now, we need to have a look for patterns for each column. The best way is to visualise the data.
#%%
import matplotlib.pyplot as plt
import seaborn as sns
#%%
df_6 = df_5.reset_index()
df_6.head()
#%% md
Let us export the data to a CSV file. We shall perform data visualisation in Tableau.
#%%
```

```
df_6.to_csv("HHS_Program_Cleaned.csv")
```

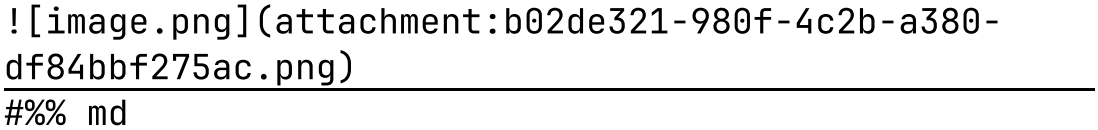
#%% md



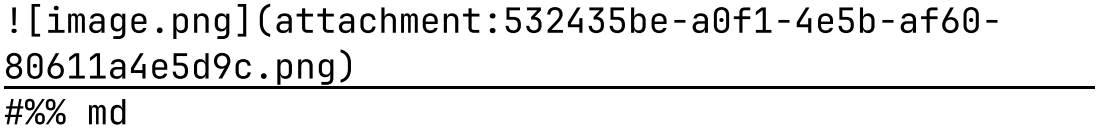
#%% md



#%% md



#%% md



#%% md

From the above graphs, we can infer that Children who were considered by CBP, those already going through CBP custody and those done with the CBP custody and transferred to HHS care follow a very similar pattern. Same is the case with the children in HHS care and those who got discharged from it.

There is another important aspect extracted from the dataset. The number of children undergoing each phase of the program has gone drastically down in February 2025, and the same level is maintained throughout the rest of the time. This is mainly due to decrease in illegal immigration to the United States, thanks to harsh restrictions set up by the Trump government. Trump came into power on January 20, 2025. Some of his initial executive orders include crackdown of illegal immigration - this is the main driver behind numbers falling down.

#%% md

Detecting Outliers using Interquartile Range Method

#%%

```
df_temp = df_6.drop(columns="Date")

Q1 = df_temp.quantile(0.25) # Lower Quartile
Q3 = df_temp.quantile(0.75) # Upper Quartile

```

```
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

df_temp[(df_temp < lower_limit) | (df_temp > upper_limit)].sum()
```

```
#%% md
As we observe, there are no values below the lower limit and above the upper limit. This means that there are no outliers at all in our dataset.
```

```
#%%
df_6.head()
```

```
#%% md
#### Renaming all columns to simple names
```

```
#%%
df_7 = df_6.copy()
df_7.rename(columns={
    "Date" : "Date",
    "Children apprehended and placed in CBP custody *" : "Placed_In_CBP",
    "Children in CBP custody" : "CBP_Custody",
    "Children transferred out of CBP custody" : "CBP_To_HHS",
    "Children in HHS Care"  : "HHS_Care",
    "Children discharged from HHS Care": "Discharged_From_HHS"
},inplace=True)

df_7.head()
```

```
#%% md
#### Creating lags and rolling averages for each column
```

```
#%%
# Retrieving all parameters of children
list_of_columns = df_7.columns.values.tolist()[1:]

# New Lag Columns
lag_by_1 = [i+"_lag_1" for i in list_of_columns]
lag_by_7 = [i+"_lag_7" for i in list_of_columns]
lag_by_14 = [i+"_lag_14" for i in list_of_columns]
```

```
182
183 # Columns for Rolling averages
184 rolling_avg_7 = [i+"_rolling_avg_7" for i in
    list_of_columns]
185 rolling_avg_14 = [i+"_rolling_avg_14" for i in
    list_of_columns]
186
187 # Calculating lag features
188 df_7[lag_by_1] = df_7[list_of_columns].shift(1)
189 df_7[lag_by_7] = df_7[list_of_columns].shift(7)
190 df_7[lag_by_14] = df_7[list_of_columns].shift(14)
191
192 # Calculating rolling averages
193 df_7[rolling_avg_7] = df_7[list_of_columns].rolling(
    7).mean()
194 df_7[rolling_avg_14] = df_7[list_of_columns].rolling
    (14).mean()
195 #%%
196 df_7.head(5)
197 #%% md
198 #### Defining Flow based signals
199 #%%
200 # Defining the net flow of children right from
    entering HHS care to discharging them
201 df_7["Net_HHS_Flow"] = df_7["CBP_To_HHS"] - df_7["
    Discharged_From_HHS"]
202
203 # Rolling averages of Net HHS
204 df_7["Net_HHS_Flow_rolling_avg_7"] = df_7["
    Net_HHS_Flow"].rolling(7).mean()
205 df_7["Net_HHS_Flow_rolling_avg_14"] = df_7["
    Net_HHS_Flow"].rolling(14).mean()
206
207 # Rolling standard deviation across 7 days
208 df_7["Net_HHS_Flow_rolling_std_7"] = df_7["
    Net_HHS_Flow"].rolling(7).std()
209
210
211 df_7.head()
212 #%% md
213 Flow acceleration is just an additional and a useful
```

```
 feature, but not mandatory. It is just to determine how inflow pressure changes as days pass on
#%% md
#### Adding calender effects
#%%
# Day of Week
df_7["Day_Of_Week"] = df_7["Date"].dt.dayofweek

# Month
df_7["Month"] = df_7["Date"].dt.month

# If the day is weekend or not
df_7["Is_Weekend"] = (df_7["Day_Of_Week"] >= 5).astype("int64")

df_7.head(8)
#%% md
#### Time-Series Decomposition
#%%
decomp = seasonal_decompose(
    x = df_7["HHS_Care"],
    model = "additive",
    period = 7
)
#%%
decomp_df = pd.DataFrame(columns=["Date","Observed","Seasonal","Trend","Residual"])
decomp_df["Date"] = df_7["Date"].copy()
decomp_df["Observed"] = decomp.observed
decomp_df["Seasonal"] = decomp.seasonal
decomp_df["Residual"] = decomp.resid
decomp_df["Trend"] = decomp.trend

decomp_df.head()
#%%
decomp_df.to_csv("HHS_decomposition.csv")
#%% md
![image.png](attachment:50639733-2b7a-49bb-8b5e-40f5084117e6.png)
#%%
df_7.head()
```

```
250 #%% md
251 #### Train test strategy
252 #%% md
253 We will not be using traditional train-test-split in
     this project, because it is a time-series dataset.
    In such datasets, we use past and present data in
    order to predict future data.
254
255 We will be using walk-forward validation technique.
    It is a validation technique that uses past data to
    predict future ones, and past data is moved forward
    by some time and then future data is predicted. For
    example, the model is trained from Jan 1 2025 to
    December 1 2025 and tested on Jan 1 2026 to Mar 31
    2026. Then we shift the thresholds by 1 week - now
    model is trained from Jan 7 2025 to Jan 7 2026, and
    tested on Jan 8 2026 to April 7 2026.
256 #%% md
257 We would be using expanding window technique of walk
     forward validation. Each time we will expand the
    training set by some time and expand the testing
    input values by the same time. For example, the
    model is trained from Jan 1 2025 to December 1 2025
    and tested on Jan 1 2026 to Mar 31 2026. Then we
    roll the training set by a week - now it is trained
    on Jan 7 2025 to Jan 7 2026. Testing set ranges from
     Jan 8 2026 to April 7 2026. This is the rolling
    window or conveyor belt method for validation.
258
259 There is another method of walk-forward validation,
    which is none other than expanding window method.
    Let us take an example. A model is trained from Jan
    1 2025 to December 1 2025 and tested on Jan 1 2026
    to Jan 7 2026. Then we expand the training set - now
     it is trained on Jan 1 2026 to Jan 7 2026. Testing
    set ranges from Jan 8 2026 to Jan 15 2026.
260 #%% md
261 We need to split the dataset to training set,
    validation window and testing set. The following are
     the best percentage shares of the dataset:
262
```

```
263 * Training set - 67%
264 * Validation Window - 25%
265 * Testing set - 8%
266 #%%
267 df_7.to_csv("final_dataset.csv",index=False)
268 #%%
269 initial_day = df_7.loc[0,"Date"]
270 final_day = df_7.loc[df_7.shape[0]-1,"Date"]
271
272 td = (final_day - initial_day)
273 total_number_of_days = td.days
274
275 number_of_training_dates = round(
    total_number_of_days * 67 / 100)
276 size_of_validation_window = round(
    total_number_of_days * 25 / 100)
277 number_of_testing_dates = total_number_of_days - (
    number_of_training_dates + size_of_validation_window
    )
278
279 training_threshold = initial_day + timedelta(
    number_of_training_dates - 1)
280 validation_threshold = training_threshold +
    timedelta(size_of_validation_window - 1)
281
282 training_dates = pd.date_range(
283     initial_day,
284     training_threshold,
285     inclusive = "left",
286 )
287
288 validation_window_1 = pd.date_range(
289     training_threshold,
290     validation_threshold,
291     inclusive = "left"
292 )
293
294 testing_dates = pd.date_range(
295     validation_threshold,
296     final_day,
297     inclusive = "both"
```

```
)
```

#%% md

The above code retrieves the training, validation, and testing dates. During walk-forward validation, we need to generate predictions for the validation data **window by window** rather than predicting one day at a time. A **validation window** refers to a fixed range of consecutive dates that the model predicts at each iteration.

For example, if we use a **7-day validation window**, we first train the model using the available training data. We then use the trained model to predict the first 7 days of the validation period. After these 7 days have passed, their **actual observed values** are added to the training data. The validation window is then moved forward by 7 days, and the model is retrained using the expanded training data before predicting the next 7 days.

This process is repeated until the entire validation period has been covered. Therefore, instead of iterating through the validation dates individually, we divide the validation dates into groups containing number of consecutive dates equal to window size. This allows us to process the validation period **window by window**, which accurately represents the walk-forward forecasting process. The following function is used to divide the validation data into fixed-size windows.

#%%

```
def retrieve_validation_window(validation_window_1, length_of_window):
    validation_data = []
    v = 0
    p = []

    for i in validation_window_1:
        if v == length_of_window:
            t = pd.date_range(p[0],p[-1])
            validation_data.append(t)
```

```
            p = []
            v = 0
        p.append(i)
        v += 1

    if p is not None:
        t = pd.date_range(p[0],p[-1])
        validation_data.append(t)

    return validation_data
#%% md
#### Training Models
#%% md
#### 1) Baseline Models
#%% md
#####   a) Naive approach
#%%
def naive_model(df,start_date,target_variable):
    predicted_value = df[df["Date"] == start_date][f"{target_variable}_lag_1"].values[0]
    return predicted_value
#%%
def multi_horizon_evaluation(df_7, validation_window_1,target_variable,method, size_of_window=7):
    validation_data = retrieve_validation_window(validation_window_1,size_of_window)
    cols = ["Day " + str(i) for i in range(1, size_of_window + 1)]

    mae_df = pd.DataFrame(columns=["Fold"] + cols)
    rmse_df = pd.DataFrame(columns=["Fold"] + cols)

    rt = 0

    for i in validation_data:
        start_date = i[0]
        df_train_test = pd.DataFrame(columns=["Date", f"Actual_{target_variable}",f"Predicted_{target_variable}"])
        # Generate predictions
```

```
349         if method == "naive":
350             target_variable_predicted_value =
    naive_model(df_7,start_date,target_variable)
351         elif method == "moving average":
352             target_variable_predicted_value =
    calculate_moving_avg(df_7,i,size_of_window,
    target_variable)
353         else:
354             raise ValueError("Invalid method. Use '
    naive' or 'moving average'.")
355 
356         # Actual and predicted values
357         df_train_test["Date"] = i
358         df_train_test[f"Actual_{target_variable}"
    ] = df_7[df_7["Date"].isin(i)][target_variable].
    values
359 
360         df_train_test[f"Predicted_{target_variable}"
    ] = target_variable_predicted_value
361 
362         cols_1 = ["Day "+str(j) for j in range(1,len
    (i)+1)]
363 
364         # Calculate absolute errors
365         df_train_test["Absolute_Error"] = abs(
    df_train_test[f"Predicted_{target_variable}"]-
    df_train_test[f"Actual_{target_variable}"])
366 
367         mae = mean_absolute_error(df_train_test[f"
    Actual_{target_variable}"],df_train_test[f"
    Predicted_{target_variable}"])
368 
369         mae_df.loc[rt, "Fold"] = rt + 1
370         mae_df.loc[rt, cols_1] = df_train_test["
    Absolute_Error"].values
371         mae_df.loc[rt, "MAE"] = round(mae,2)
372 
373         # Calculate squared errors
374         df_train_test["Squared_Error"] = (
    df_train_test[f"Predicted_{target_variable}"] -
    df_train_test[f"Actual_{target_variable}"]) ** 2
```

```
375
376         rmse = root_mean_squared_error(df_train_test
    [f"Actual_{target_variable}"],df_train_test[f"
    Predicted_{target_variable}"])
377
378         rmse_df.loc[rt, "Fold"] = rt + 1
379         rmse_df.loc[rt, cols_1] = df_train_test["
    Squared_Error"].values
380         rmse_df.loc[rt, "RMSE"] = round(rmse,2)
381
382         rt += 1
383
384     # MAE by forecast horizon
385     mae_horizon_error_profile = mae_df.mean()
386     mae_horizon_error_profile.drop(["Fold", "MAE"],
    axis=0,inplace=True)
387     mae_horizon_error_profile =
    mae_horizon_error_profile.round(2)
388
389     # RMSE by forecast horizon
390     rmse_horizon_error_profile = rmse_df.mean()
391     rmse_horizon_error_profile.drop(["Fold", "RMSE"
    ],axis=0,inplace=True)
392     rmse_horizon_error_profile = np.power(
    rmse_horizon_error_profile,0.5).round(2)
393
394     return (mae_df,rmse_df,mae_horizon_error_profile
    ,rmse_horizon_error_profile)
395 #%%
396 (
397     mae_df,
398     rmse_df,
399     mae_horizon_error_profile,
400     rmse_horizon_error_profile
401 ) = multi_horizon_evaluation(
402     df_7,
403     validation_window_1,
404     "HHS_Care",
405     "naive",
406     size_of_window=7
407 )
```

```
#%%
mae_horizon_error_profile
#%%
rmse_horizon_error_profile
#%% md
##### b) Moving average forecast
#%%
def calculate_moving_avg(df, window_dates, window_size, target_variable):
    return df[df["Date"].isin(window_dates)][f"{target_variable}_rolling_avg_{window_size}"].round(2).values
#%%
(
    mae_df,
    rmse_df,
    mae_horizon_error_profile,
    rmse_horizon_error_profile
) = multi_horizon_evaluation(
    df_7,
    validation_window_1,
    "HHS_Care",
    "moving average",
    size_of_window=7
)
#%%
mae_horizon_error_profile
#%%
rmse_horizon_error_profile
#%% md
#### 2) Statistical Models
#%% md
##### a) ARIMA
#%%
df_7.head()
#%%
df_i = df_7[["Date","HHS_Care"]]
df_i["Date"] = pd.to_datetime(df_i["Date"])
df_i.head(10)
#%%
sns.lineplot(x=df_i["Date"],y=df_i["HHS_Care"])
```

```
#%%
result = adfuller(df_i["HHS_Care"])
adf = result[0]
p_value = result[1]
print("Stationarity measure (p-value): ",round(p_value,2))
if p_value > 0.05:
    print("Series is not stationary, we need to perform differencing")
else:
    print("Series is stationary, no need to perform differencing")
```

```
#%% md
Now, we see that the series is not stationary and that differencing needs to be applied on HHS Care.
```

```
#%%
df_i["HHS_Care_diff_1"] = df_i["HHS_Care"].diff()
df_j = df_i.dropna()
sns.lineplot(x=df_j["Date"],y=df_j["HHS_Care_diff_1"])
```

```
#%%
result_2 = adfuller(df_j["HHS_Care_diff_1"])
adf_2 = result_2[0]
p_value_2 = result_2[1]

print("Stationarity measure (p-value): ",round(p_value_2,2))
if p_value_2 > 0.05:
    print("Series is not stationary, we need to perform differencing")
else:
    print("Series is stationary, no need to perform differencing")
```

```
#%% md
The following is the function that performs repeated differencing until stationarity is achieved. It returns the differenced dataset and name of the column which has all resultant stationary data
```

```
#%%
def differencing_method(df,target_variable):
    df_i = df.copy()
```

```
    i = 0
    while True:
        result = adfuller(df_i[f"{target_variable}_diff_"+str(i)])
        adf = result[0]
        p_value = result[1]
        print("Stationarity measure (p-value): ", round(p_value,2))
        if p_value <= 0.05:
            print("Series is stationary, no need to perform differencing")
            break
        else:
            print("Series is not stationary, we need to perform differencing")
            df_i[f"{target_variable}_diff_"+str(i+1)] = df_i[f"{target_variable}_diff_"+str(i)].diff()
            df_i = df_i.dropna()
            i += 1
            continue
    d = i
    return df_i,f"{target_variable}_diff_"+str(i),d
```

```
#%%
def perform_differencing(df_7,target_variable):
    df_i = df_7[["Date", target_variable]]
    df_i[f"{target_variable}_diff_0"] = df_i[target_variable].copy()
    df_i["Date"] = pd.to_datetime(df_i["Date"])
    df_i, name_of_stationary_column,d = differencing_method(df_i,target_variable)
    return df_i, name_of_stationary_column,d
```

```
#%%
df_i,name_of_stationary_column,d = perform_differencing(df_7,"HHS_Care")
df_i.head()
```

```
#%% md
Now we are going to plot Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots. PACF plot is used for check for number of Autoregressive (AR) terms (p) and ACF is used for number of previous forecast errors the current
```

```
forecasted value is dependent on.
#%%
ax = plot_pacf(df_i[name_of_stationary_column])
#%% md
As we have a look at the above graph, we observe that there are significant spikes at spikes from 0 to 7. Hence, we shall consider values of p from 1 to 7 for training our ARIMA model.
#%%
ax2 = plot_acf(df_i[name_of_stationary_column])
#%% md
The above autocorrelation function graph depicts a . And moreover, the spikes are significant at q = 1,2,5,6,7,8. Hence we would set q value from 1 to 8.


The above autocorrelation function comprises spikes that are significant at q = 1,2,5,6,7,8. Hence we would set q value from 1 to 8. We also see a pattern that repeats every 7 days. This means that it is better if we use the Seasonal Autogressive Integrated Moving Average (Seasonal ARIMA - SARIMA) model instead of normal ARIMA. Hence, we shall use both ARIMA and SARIMA to check which one produces best results.
#%% md
The following is the function that creates and returns an ARIMA model for given training dataset (df), number of autoregressive (p) and number of moving average (q) terms.
#%%
def find_p_and_q_values(df_i, name_of_stationary_column, max_lags=30, alpha=0.05):
    series = df_i[name_of_stationary_column].dropna()
    max_lags = min(max_lags, len(series)//2-1)

    if max_lags < 1:
        raise ValueError("Insufficient observations to calculate PACF and ACF.")

```

```
526     acf_values = acf(series, nlags=max_lags, alpha=
    alpha)
527     pacf_values = pacf(series, nlags=max_lags, alpha
    =alpha, method="ywm")
528 
529     acf_coefficients = acf_values[0]
530     acf_confidence = acf_values[1]
531 
532     pacf_coefficients = pacf_values[0]
533     pacf_confidence = pacf_values[1]
534 
535     acf_significant_lags = [lag for lag in range(1,
    max_lags+1) if not (acf_confidence[lag, 0]<=0<=
    acf_confidence[lag, 1])]
536 
537     pacf_significant_lags = [lag for lag in range(1,
    max_lags+1) if not (pacf_confidence[lag, 0]<=0<=
    pacf_confidence[lag, 1])]
538 
539     p = pacf_significant_lags[0:3]
540     q = acf_significant_lags[0:3]
541 
542     # Display significant lags
543     print("Significant PACF lags (candidate q values
    ):",
544           p)
545 
546     print("Significant ACF lags (candidate p values
    ):",
547           q)
548 
549     return p,q
```

```
550 #%%
551 p,q = find_p_and_q_values(df_i,
    name_of_stationary_column)
```

```
552 #%%
553 def retrieve_arima_model(df,p,d,q,target_variable):
554     try:
555         model = ARIMA(df[target_variable],order=(p,d
    ,q))
556         return model
```

```
    except:
        return 0
```

```
#%% md
```

We need to perform walk-forward validation on number of children in HHS Care (HHS Care column) across the validation dataset. In this case, we will consider the validation window of one week - 7 days.

In the first iteration, we will consider the entire training dataset for HHS Care column, and predict the values for first validation window (first 7 values in the validation set). For the second iteration, we will consider the training dataset + first validation window. Prediction is carried out on second validation window (second 7 values in the validation set. For third iteration, we consider training data + first validation window + second validation window for training, and third validation window for prediction. Likewise, we keep rolling forward throughout the validation dates by considering each window of 7 days until we reach the end of the validation dataset. This entire process is carried out for each pair of p and q values.

Create two dataframes - one which comprises mean absolute errors and the other with root mean squared errors.

For mean absolute error: The columns include the (p, q) value pair, mean absolute errors calculated on actual and predicted values of each validation window during walk-forward validation; and the average of mean absolute error across all windows in each (p,q) order.

Similarly, for root mean squared error: The columns include the (p,q) value pair, root mean squared errors calculated on actual and predicted values of each validation window during walk-forward validation; and the average of root mean squared errors across all windows in each (p,q) order.

```
569 #%%
570 def multi_horizon_arima_evaluation(df_7,
    target_variable,p,d,q,training_dates,
    validation_window_1,size_of_window=7):
571     validation_data = retrieve_validation_window(
    validation_window_1, size_of_window)
572     no_of_validation_windows = len(validation_data)
573     cols = ["Window " + str(i + 1) for i in range(
    no_of_validation_windows)]
574     cols_mae = ["Model"] + cols + ["Average MAE"]
575     cols_rmse = ["Model"] + cols + ["Average RMSE"]
576 
577     df_arima_mae = pd.DataFrame(columns=cols_mae)
578     df_arima_rmse = pd.DataFrame(columns=cols_rmse)
579 
580     list_of_models = []
581     no_of_validation_dates_for_models = []
582 
583     ctr = 0
584     combination_of_p_and_q_values = list(itertools.
    product(p, q))
585 
586     df_arima_mae["Model"] =
    combination_of_p_and_q_values.copy()
587     df_arima_rmse["Model"] =
    combination_of_p_and_q_values.copy()
588 
589     for i in combination_of_p_and_q_values:
590         for num in range(0, len(validation_data)):
591             no_of_steps = len(validation_data[num])
592             # print(no_of_steps)
593             df_w = df_7[
594                 [k in training_dates or k in
    validation_window_1[0:size_of_window * num] for k in
     df_7["Date"]]].copy()
595             # model,AIC,order = retrieve_arima_model
    (df_w)
596             model = retrieve_arima_model(df_w, i[0
    ], d, i[1], target_variable)
597             result = model.fit()
598             predicted_values = result.forecast(steps
```

```
598 =no_of_steps)
599             list_of_models.append(model)
600             no_of_validation_dates_for_models.append
    (size_of_window * num)
601 
602             mae = mean_absolute_error(df_7.loc[
    predicted_values.index.values, target_variable],
    predicted_values)
603             rmse = root_mean_squared_error(df_7.loc[
    predicted_values.index.values, target_variable],
    predicted_values)
604 
605             df_arima_mae.loc[ctr, "Window " + str(
    num + 1)] = mae
606             df_arima_rmse.loc[ctr, "Window " + str(
    num + 1)] = rmse
607         ctr += 1
608         print(ctr)
609 
610     for i in df_arima_mae.index:
611         df_arima_mae.loc[i, "Average MAE"] =
    df_arima_mae.loc[i, cols].mean()
612         df_arima_rmse.loc[i, "Average RMSE"] =
    df_arima_rmse.loc[i, cols].mean()
613 
614     position_of_best_model = int(np.where(
    df_arima_mae["Average MAE"] == df_arima_mae["Average
     MAE"].min())[0][0])
615     p1, q1 = df_arima_mae.loc[position_of_best_model
    , "Model"]
616     return p1,q1,df_arima_mae,df_arima_rmse
617 #%%
618 p1, q1, df_arima_mae,df_arima_rmse =
    multi_horizon_arima_evaluation(
619     df_7=df_7,
620     target_variable="HHS_Care",
621     p=p,
622     d=d,
623     q=q,
624     training_dates=training_dates,
625     validation_window_1=validation_window_1,
```

```
    size_of_window=7
)
#%%
p1,q1
#%% md
We then figure out the (p,q) pair that returns the best models from the MAE and RMSE dataframes - The one candidate which gave the least MAE and RMSE is considerd the best model candidate, and their corresponding (p,q) pair is called the best pair.
#%%
position_of_best_model = int(np.where(df_arima_mae["Average MAE"] == df_arima_mae["Average MAE"].min())[0][0])

p,q = df_arima_mae.loc[position_of_best_model,"Model"]

df_arima_train = df_7[[k in training_dates or k in validation_window_1 for k in df_7["Date"]]].copy()

arima_model = retrieve_arima_model(df_arima_train,p,d,q,"HHS_Care")

fit_arima_model = arima_model.fit()

print(position_of_best_model)
print((p,q))
#%%
df_arima_mae
#%% md
Now, using the (p,q) value pair for which we found the best model candidate, we create a new model and train it with both training and validation datasets combined. We then fit the model and find the residuals - Training errors - Differences between each corresponding actual and predicted values in training and validation datasets
#%%
residuals = fit_arima_model.resid
residuals
```

```
#%%
plt.plot(residuals)
#%% md
We see that the first residual is unexpectedly large and all the residuals fluctuate around zero. This means that we need to make an investigation on the first residual. This happens due to application of differencing before training of ARIMA model. We need to check if this happened due to some initialisation effect or because of the model/ dataset.
#%% md
Step 1 - Check the first few residuals
#%%
residuals.head(10)
#%% md
From this, we can say the value of first residual is unexpectedly high. The other values are between 20 and 30.
#%% md
Step 2 - Plot without the first residual
#%%
plt.plot(residuals[1:])
#%% md
Now, since we excluded the first residual, we observe that the values hover around zero. This can be a sign of encouragement - because of the random noise pattern.
#%% md
Step 3 - Plot the ACF plot and apply the Ljung-Box test on residuals
#%% md
Residual ACF
#%%
plot_acf(residuals)
#%% md
Ljung-Box test
#%%
ljung_box = acorr_ljungbox(
    residuals,
    lags = [7],
```

```
    return_df = True
)
print(ljung_box)
#%% md
From the ACF plot, we see that the ACF at 0 is 1, which is completely normal. We also carried out the Ljung-Box test, which gave us the p-value of 0.84, which is greater than 0.05. So the ARIMA model that we trained can be subjected to final forecasting.
#%%
df_arima_test = df_7[[k in testing_dates for k in df_7["Date"]]].copy().reset_index(drop=True)

df_arima_test_hhs_care = df_arima_test["HHS_Care"].values
df_arima_predicted_values = fit_arima_model.forecast(steps=df_arima_test.shape[0])

error_of_arima_model = root_mean_squared_error(df_arima_test_hhs_care,df_arima_predicted_values)

print("Final error of ARIMA model : ", error_of_arima_model)
#%% md
The above is the root mean squared error that we got due to comparison of actual number of children in HHS care with forecasted values from the testing dataset.
#%% md
The following is the multi-horizon evaluation that we perform on validation dataset for ARIMA model:
#%%
def retrieve_arima_model(df,p,d,q,target_variable):
    try:
        model = ARIMA(df[target_variable],order=(p,d,q))
        return model
    except:
        return 0
#%%
def calculate_arima_model(df,size_of_window,p,d,q,
```

```
706 target_variable):
707     arima_model = retrieve_arima_model(df,p,d,q,
    target_variable)
708     final_model = arima_model.fit()
709     predictions = final_model.forecast(steps=
    size_of_window)
710     return predictions
711 #%%
712 def multi_horizon_arima_error_profile(df,
    target_variable,training_dates,validation_window_1,p
    ,d,q,size_of_window=7):
713     validation_data = retrieve_validation_window(
    validation_window_1,size_of_window)
714 
715     cols = ["Day " + str(i) for i in range(1,
    size_of_window + 1)]
716 
717     mae_df = pd.DataFrame(columns=["Fold"] + cols)
718     rmse_df = pd.DataFrame(columns=["Fold"] + cols)
719 
720     validation_dates = []
721 
722     for rt, validation_window in enumerate(
    validation_data):
723         no_of_steps = len(validation_window)
724         cols_1 = ["Day " + str(i) for i in range(1,
    no_of_steps + 1)]
725 
726         df_w = df.loc[df["Date"].isin(list(
    training_dates) + validation_dates)].copy()
727 
728         df_w = df_w.sort_values("Date")
729 
730         predictions = calculate_arima_model(df_w,
    no_of_steps,p,d,q,target_variable)
731 
732         df_c = pd.DataFrame()
733         df_c["Actual"] = df.loc[df["Date"].isin(
    validation_window), target_variable].values
734         df_c["Predicted"] = np.asarray(predictions)
735         df_c["Absolute Error"] = abs(df_c["Actual"
```

```
735 ] - df_c["Predicted"])
736         df_c["Squared Error"] = (df_c["Actual"] -
    df_c["Predicted"]) ** 2
737         mae = df_c["Absolute Error"].mean()
738         rmse = np.sqrt(df_c["Squared Error"].mean())
739         mae_df.loc[rt, "Fold"] = rt + 1
740         mae_df.loc[rt, cols_1] = (df_c["Absolute
    Error"].values)
741         mae_df.loc[rt, "MAE"] = round(mae, 2)
742         rmse_df.loc[rt, "Fold"] = rt + 1
743         rmse_df.loc[rt, cols_1] = (df_c["Squared
    Error"].values)
744         rmse_df.loc[rt, "RMSE"] = round(rmse, 2)
745         validation_dates.extend(validation_window)
746
747     mae_horizon_error_profile = (mae_df[cols].mean
    ()).round(2)
748
749     # Calculate RMSE for each forecast horizon
750     rmse_horizon_error_profile = np.sqrt(rmse_df[
    cols].mean()).round(2)
751
752     return (
753         mae_df,
754         rmse_df,
755         mae_horizon_error_profile,
756         rmse_horizon_error_profile
757     )
758 #%%
759 (
760     mae_df,
761     rmse_df,
762     mae_horizon_error_profile,
763     rmse_horizon_error_profile
764 ) = multi_horizon_arima_error_profile(
765     df = df_7,
766     target_variable = "HHS_Care",
767     training_dates = training_dates,
768     validation_window_1 = validation_window_1,
769     p = p,
770     d = d,
```

```
    q = q,
    size_of_window=7
)
#%%
size_of_window = 7
validation_data = retrieve_validation_window(validation_window_1,size_of_window)
cols = ["Day "+str(i) for i in range(1, size_of_window+1)]
mae_df = pd.DataFrame(columns = ["Fold"]+cols)
rmse_df = pd.DataFrame(columns = ["Fold"]+cols)
rt = 0
validation_dates = []


for i in validation_data:
    no_of_steps = len(i)

    cols_1 = ["Day "+str(i) for i in range(1, no_of_steps+1)]

    df_c = pd.DataFrame(columns=["Actual","Predicted"])

    df_c["Actual"] = df_7[[k in i for k in df_7["Date"]]]["HHS_Care"].copy()

    df_w = df_7[[k in training_dates or k in validation_dates for k in df_7["Date"]]].copy()

    predictions = calculate_arima_model(df_w, no_of_steps,1,0)
    df_c["Predicted"] = predictions

    df_c["Absolute Error"] = abs(df_c["Actual"] - df_c["Predicted"])
    df_c["Squared Error"] = (df_c["Actual"] - df_c["Predicted"]) * (df_c["Actual"] - df_c["Predicted"])

    mae = df_c["Absolute Error"].mean()
    rmse = np.sqrt(df_c["Squared Error"].mean())
```

```
803
804     mae_df.loc[rt,"Fold"] = (rt + 1)
805     mae_df.loc[rt,cols_1] = df_c["Absolute Error"].
    values
806     mae_df.loc[rt,"MAE"] = round(mae,2)
807
808     rmse_df.loc[rt,"Fold"] = (rt + 1)
809     rmse_df.loc[rt,cols_1] = df_c["Squared Error"].
    values
810     rmse_df.loc[rt,"RMSE"] = round(rmse,2)
811
812
813     rt += 1
814     validation_dates.extend(i)
815
816 mae_horizon_error_profile = mae_df.mean()
817 mae_horizon_error_profile.drop(["Fold","MAE"],axis=0
    ,inplace=True)
818 mae_horizon_error_profile =
    mae_horizon_error_profile.round(2)
819
820 rmse_horizon_error_profile = rmse_df.mean()
821 rmse_horizon_error_profile.drop(["Fold","RMSE"],axis
    =0,inplace=True)
822 rmse_horizon_error_profile = round(np.power(
    rmse_horizon_error_profile,0.5),2)
823 #%%
824 mae_df[cols].mean()
825 #%%
826 rmse_df[cols].mean()
827 #%% md
828 ##### b) Exponential Smoothing
829 #%% md
830 Before going for exponential smoothing, let us
    display the first five records of the dataset so
    that we can recall the dataset
831 #%%
832 df_7.head()
833 #%% md
834 There are three types of exponential smoothing:
835
```

```
1) Simple/Single Exponential Smoothing - Applied on data with NO trend or seasonality
2) Double Exponential Smoothing - Applied on data with trend
3) Triple Exponential Smoothing - Applied on data with both trend and seasonality

In order to check whether our target variable (HHS_Care) has trend or seasonality - we need to plot the graph of HHS_Care with respect to time. Then we can go ahead with choosing the type of exponential smoothing that suits our data points.
#%%
df_k = df_7[["Date","HHS_Care"]].set_index("Date")
df_k.head()
#%%
plt.subplots(figsize=(10,5))
plt.plot(df_k)
#%% md
From the above graph, we can notice that it is not seasonal, but has noticeable trends. We see that there are less number of children going through HHS Care fom roughly March of 2025. In this case, Double Exponential Smoothing is the suitable method - since it has trend component but not exponential.
#%%
import pandas as pd

from statsmodels.tsa.holtwinters import SimpleExpSmoothing,Holt,ExponentialSmoothing

def select_exponential_smoothing_model(df, target_variable,size_of_window=7):
    series = df.set_index("Date")[target_variable].dropna()
    fitted_models = {}
    model_results = []

    model_configurations = [
        {
            "name": "Simple Exponential Smoothing",
```

```
            "trend": None,
            "damped_trend": False,
            "seasonal": None
        },
        {
            "name": "Holt Linear Trend",
            "trend": "add",
            "damped_trend": False,
            "seasonal": None
        },
        {
            "name": "Holt Damped Trend",
            "trend": "add",
            "damped_trend": True,
            "seasonal": None
        },
        {
            "name": "Holt-Winters Additive",
            "trend": "add",
            "damped_trend": False,
            "seasonal": "add"
        },
        {
            "name": "Holt-Winters Multiplicative",
            "trend": "add",
            "damped_trend": False,
            "seasonal": "mul"
        }
    ]

    for config in model_configurations:
        name = config["name"]
        try:
            if config["seasonal"] is not None:
                if len(series) < 2 * size_of_window:
                    print(f"Skipping {name}:
    insufficient observations for seasonal
    initialization.")
                    continue

            if config["seasonal"] == "mul" and (
```

```
series <= 0).any():
                print(f"Skipping {name}:
multiplicative seasonality requires positive
observations.")
                continue


            model = ExponentialSmoothing(
                series,
                trend=config["trend"],
                damped_trend=config["damped_trend"],
                seasonal=config["seasonal"],
                seasonal_periods=(
                    size_of_window
                    if config["seasonal"] is not
None
                    else None
                ),
                initialization_method="estimated"
            )

            fitted_model = model.fit(optimized=True)

            fitted_models[name] = fitted_model

            model_results.append({
                "Model": name,
                "AIC": fitted_model.aic,
                "BIC": fitted_model.bic,
                "SSE": fitted_model.sse
            })

        except (ValueError, ArithmeticError, np.
linalg.LinAlgError) as e:
            print(f"Could not fit {name}: {e}")

    results_df = pd.DataFrame(model_results)

    if results_df.empty:
        raise ValueError("None of the exponential
smoothing models could be fitted.")
```

```
936
937     results_df = results_df.sort_values("AIC").
    reset_index(drop=True)
938     best_model_name = results_df.loc[0, "Model"]
939
940     return best_model_name
941 #%%
942 def train_exponential_smoothing_model(df,
    best_model_name,target_variable,size_of_window=7):
943     model_configurations = pd.DataFrame([
944             {
945                 "name": "Simple Exponential
    Smoothing",
946                 "trend": None,
947                 "damped_trend": False,
948                 "seasonal": None
949             },
950             {
951                 "name": "Holt Linear Trend",
952                 "trend": "add",
953                 "damped_trend": False,
954                 "seasonal": None
955             },
956             {
957                 "name": "Holt Damped Trend",
958                 "trend": "add",
959                 "damped_trend": True,
960                 "seasonal": None
961             },
962             {
963                 "name": "Holt-Winters Additive",
964                 "trend": "add",
965                 "damped_trend": False,
966                 "seasonal": "add"
967             },
968             {
969                 "name": "Holt-Winters Multiplicative
    ",
970                 "trend": "add",
971                 "damped_trend": False,
972                 "seasonal": "mul"
```

```
            }
        ])

    model_config = model_configurations[model_configurations["name"]==best_model_name]
    series = df.set_index("Date")[target_variable].dropna()

    expo_smooth = ExponentialSmoothing(
        series,
        trend=model_config["trend"].values[0],
        damped_trend=model_config["damped_trend"].values[0],
        seasonal=model_config["seasonal"].values[0],
        seasonal_periods=(size_of_window if model_config["seasonal"].values[0] is not None else None),
        initialization_method="estimated"
    )

    model = expo_smooth.fit()

    return model
#%%
best_model_name = select_exponential_smoothing_model(
        df=df_7,
        target_variable="HHS_Care",
        size_of_window=7
)
print(best_model_name)

kf = train_exponential_smoothing_model(df_7,best_model_name,"HHS_Care",size_of_window=7)
kf.forecast(7)
#%%
def multi_horizon_exponential_smoothing_evaluation(df,validation_window_1,training_dates,model_name,target_variable,size_of_window=7):

```

```
    validation_data = retrieve_validation_window(validation_window_1,size_of_window)

    max_horizon = max(
        (len(window) for window in validation_data),
        default=0
    )

    #print(validation_data)

    cols = [f"Day {i}" for i in range(1, max_horizon + 1)]

    mae_df = pd.DataFrame(columns=["Fold"] + cols)
    rmse_df = pd.DataFrame(columns=["Fold"] + cols)

    # Track previously observed validation dates
    validation_list = []
    ut = []
    lk = []

    for fold, validation_window in enumerate(validation_data):

        # Construct training dataset
        train_mask = (
            df["Date"].isin(training_dates)
            | df["Date"].isin(validation_list)
        )


        df_repeated_train = df.loc[train_mask].copy()

        # Fit the Exponential Smoothing model
        exp_smooth_fitted = train_exponential_smoothing_model(
            df_repeated_train,
            model_name,
            target_variable,
```

```
1039             size_of_window=size_of_window
1040         )
1041 
1042         # Forecast the validation window
1043         forecasted_values = exp_smooth_fitted.
     forecast(
1044             len(validation_window)
1045         )
1046 
1047         # Retrieve actual values in validation-
     window order
1048         actual_values = (
1049             df.loc[df["Date"].isin(
     validation_window)]
1050             .set_index("Date")
1051             .reindex(validation_window)[
     target_variable]
1052             .to_numpy()
1053         )
1054         ut.append(df.loc[df["Date"].isin(
     validation_window)]
1055             .set_index("Date")
1056             .reindex(validation_window)[
     target_variable]
1057             .to_numpy())
1058 
1059         predicted_values = np.asarray(
     forecasted_values)
1060 
1061         # Calculate errors
1062         absolute_errors = np.abs(actual_values -
     predicted_values)
1063         squared_errors = (actual_values -
     predicted_values) ** 2
1064 
1065         # Horizon-specific columns
1066         horizon_cols = [
1067             f"Day {i}"
1068             for i in range(1, len(validation_window
     ) + 1)
1069         ]
```

```
1070
1071          # Store fold number
1072          mae_df.loc[fold, "Fold"] = fold + 1
1073          rmse_df.loc[fold, "Fold"] = fold + 1
1074
1075          # Store horizon-specific errors
1076          mae_df.loc[fold, horizon_cols] = np.round(
1077              absolute_errors, 2
1078          )
1079
1080          rmse_df.loc[fold, horizon_cols] = np.round(
1081              squared_errors, 2
1082          )
1083
1084          # Calculate fold-level MAE and RMSE
1085          mae_df.loc[fold, "MAE"] = np.round(
1086              np.mean(absolute_errors), 2
1087          )
1088
1089          rmse_df.loc[fold, "RMSE"] = np.round(
1090              np.sqrt(np.mean(squared_errors)), 2
1091          )
1092
1093          # Add observed validation dates for
     subsequent folds
1094          validation_list.extend(validation_window)
1095
1096      # Average MAE for each forecast horizon
1097      mae_horizon_error_profile = (
1098          mae_df[cols].mean(axis=0).round(2)
1099      )
1100
1101      # RMSE for each forecast horizon across folds
1102      rmse_horizon_error_profile = (
1103          np.power(rmse_df[cols].mean(axis=0),0.5).
     round(2)
1104      )
1105
1106      return (
1107          mae_df,
1108          rmse_df,
```

```
        mae_horizon_error_profile,
        rmse_horizon_error_profile
    )
#%%
(
    mae_df,
    rmse_df,
    mae_horizon_error_profile,
    rmse_horizon_error_profile
) = multi_horizon_exponential_smoothing_evaluation(
    df=df_7,
    validation_window_1=validation_window_1,
    training_dates=training_dates,
    model_name=best_model_name,
    target_variable="HHS_Care",
    size_of_window=7
)
#%%
mae_horizon_error_profile
#%%
rmse_horizon_error_profile
#%% md
#### 3) Machine Learning Models
#%% md
##### a) Random Forest Regressor
#%%
df_7.head()
#%% md
In Random forest regression, we do not predict
    future values using past values like we did for
    previous models. In other words, it is not a self-
    supervised model (The above four models are self-
    supervised models, because the next prediction is
    based on previous values of the same variable). The
     output label depends on input features. This means
     that output depends on certain inputs.

In this project, the number of children going
    through HHS Care is dependent on the previous
    values, which denotes the need for self-supervised
    learning. But Random forest is not a self-
```

supervised model. Hence, we need to convert the self-supervised approach into supervised approach in which the values on previous days will be considered as separate input features.

We have already calculated the lag values of the other variables (Placed_in_CBP, CBP_Custody, CBP_to_HHS, HHS_Care and Discharged_From_HHS) and stored them in separate - we are going to use them as input features.

We would not be considering the lag features for Net HHS Flow - this is because Net HHS Flow is the difference of CBP_To_HHS and Discharged_From_HHS, which we are already using as inputs for our Random Forest Regression. Inclusion of this measure results in overfitting.

```
#%%
def consider_only_lag_columns(df_7):
    all_columns = pd.Series(df_7.columns.to_numpy())

    placed_in_cbp_lags = all_columns[all_columns.str.startswith("Placed_In_CBP")].values.tolist()
    cbp_custody_lags = all_columns[all_columns.str.startswith("CBP_Custody")].values.tolist()
    cbp_to_hhs_lags = all_columns[all_columns.str.startswith("CBP_To_HHS")].values.tolist()
    hhs_care_lags = all_columns[all_columns.str.startswith("HHS_Care")].values.tolist()
    discharged_from_hhs_lags = all_columns[all_columns.str.startswith("Discharged_From_HHS")].values.tolist()

    lag_columns = ["Date"] + placed_in_cbp_lags + cbp_custody_lags + cbp_to_hhs_lags + hhs_care_lags + discharged_from_hhs_lags + ["Day_Of_Week", "Month", "Is_Weekend"]

    df_lag = df_7[lag_columns]

```

```
1158     df_lag.fillna(0,inplace=True)
1159 
1160     return df_lag
1161 
1162 df_lag = consider_only_lag_columns(df_7)
1163 #%% md
1164 As we observe the above dataset, we see that there
     are null values. This is because the HHS Care
     program was not conducted on days before the start
     date. Hence, we need to replace null values with
     zero.
1165 #%%
1166 from sklearn.ensemble import RandomForestRegressor
1167 #%%
1168 def retrieve_random_forest_model(df,target_variable
     ):
1169     X = df.drop(columns=[target_variable])
1170     y = df[target_variable]
1171     random_forest_regr = RandomForestRegressor()
1172     random_forest_model = random_forest_regr.fit(X,
     y)
1173     return random_forest_model
1174 #%%
1175 def multi_horizon_ensemble_error_profile(df_lag,
     validation_window_1,target_variable,ml_algo,
     size_of_window=7):
1176 
1177     # Initialize evaluation DataFrames
1178     cols = ["Day " + str(i) for i in range(1,
     size_of_window + 1)]
1179 
1180     df_lag_train = df_lag[[k in training_dates for
     k in df_lag["Date"]]].copy()
1181 
1182     df_lag_valid = df_lag[[k in validation_window_1
      for k in df_lag["Date"]]].copy()
1183 
1184     validation_data = retrieve_validation_window(
     validation_window_1,size_of_window)
1185 
1186     mae_df = pd.DataFrame(columns=["Fold"] + cols)
```

```
    rmse_df = pd.DataFrame(columns=["Fold"] + cols)

    # Store validation dates already processed
    processed_valid_dates = []

    # Store actual and predicted values
    uf = pd.DataFrame(
        columns=["Date", "Actual", "Predicted"]
    )

    t = 0

    for i in validation_data:

        # Retrieve previously processed validation dates
        df_lag_valid_dates = df_lag_valid[
            df_lag_valid["Date"].isin(processed_valid_dates)
        ].copy()

        # Combine training data with previously processed
        # validation data
        df_input = pd.concat(
            [df_lag_train, df_lag_valid_dates],
            axis=0
        ).drop(columns="Date")

        # Train the Random Forest model
        if ml_algo == "random forest":
            model = retrieve_random_forest_model(
                df_input,
                target_variable
            )
        elif ml_algo == "gradient boosting":
            model = retrieve_gradient_boosting_model(
                df_input,
                target_variable
            )
```

```
1224         else:
1225             raise ValueError("Invalid method. Use '
     random forest' or 'gradient boosting'.")
1226 
1227         # Retrieve the current validation window
1228         test = df_lag_valid[
1229             df_lag_valid["Date"].isin(i)
1230         ].copy()
1231 
1232         # Separate features and target
1233         X_test = test.drop(
1234             columns=["Date", target_variable]
1235         )
1236 
1237         y_test = test[target_variable]
1238 
1239         # Generate predictions
1240         y_pred = model.predict(X_test)
1241 
1242         # Create prediction DataFrame
1243         po = pd.DataFrame({
1244             "Date": test["Date"].values,
1245             "Actual": y_test.values,
1246             "Predicted": y_pred
1247         })
1248 
1249         # Calculate errors
1250         po["Absolute Error"] = np.abs(
1251             po["Actual"] - po["Predicted"]
1252         )
1253 
1254         po["Squared Error"] = (
1255             po["Actual"] - po["Predicted"]
1256         ) ** 2
1257 
1258         # Generate horizon column names for the
     current window
1259         l = len(y_pred)
1260         cols = ["Day " + str(j) for j in range(1, l
      + 1)]
1261 
```

```
1262         # Store fold number
1263         mae_df.loc[t, "Fold"] = t + 1
1264         rmse_df.loc[t, "Fold"] = t + 1
1265 
1266         # Store daily absolute and squared errors
1267         mae_df.loc[t, cols] = po["Absolute Error"].
     values
1268         rmse_df.loc[t, cols] = po["Squared Error"].
     values
1269 
1270         # Calculate fold-level MAE and RMSE
1271         mae_df.loc[t, "MAE"] = (
1272             po["Absolute Error"].mean().round(2)
1273         )
1274 
1275         rmse_df.loc[t, "RMSE"] = np.round(
1276             np.sqrt(po["Squared Error"].mean()), 2
1277         )
1278 
1279         # Append predictions and errors
1280         uf = pd.concat(
1281             [uf, po],
1282             axis=0,
1283             ignore_index=True
1284         )
1285 
1286         # Mark current validation dates as
     processed
1287         processed_valid_dates.extend(i)
1288 
1289         t += 1
1290 
1291     # Calculate horizon-wise MAE profile
1292     mae_horizon_error_profile = (
1293         mae_df.drop(columns=["Fold", "MAE"], errors
     ="ignore")
1294         .mean()
1295         .round(2)
1296     )
1297 
1298     # Calculate horizon-wise RMSE profile
```

```
    rmse_horizon_error_profile = np.power(
        rmse_df.drop(
            columns=["Fold", "RMSE"],
            errors="ignore"
        ).mean(), 0.5
    ).round(2)

    # Return evaluation results
    return (
        mae_df,
        rmse_df,
        mae_horizon_error_profile,
        rmse_horizon_error_profile
    )
```

```
#%%
size_of_window = 7
df_lag_train = df_lag[[k in training_dates for k in
 df_lag["Date"]]].copy()

df_lag_valid = df_lag[[k in validation_window_1 for
 k in df_lag["Date"]]].copy()

df_lag_test = df_lag[[k in testing_dates for k in
df_lag["Date"]]].copy()

(
    mae_df,
    rmse_df,
    mae_horizon_error_profile,
    rmse_horizon_error_profile
) = multi_horizon_ensemble_error_profile(
    df_lag,
    validation_window_1,
    "HHS_Care",
    "random forest",
    size_of_window=7
)
```

```
#%%
mae_horizon_error_profile
```

```
#%%
rmse_horizon_error_profile
```

```
#%% md
##### b) Gradient Boosting Regressor
#%% md
Gradient Boosting Regressor also uses decision trees, just like Random Forest Regressor. The only difference is that it build trees one after the other just to reduce the prediction errors made by previous tree(s). We perform the same steps as the Random Forest Regressor here.

We have already found out what input features we needed to consider and also created new dataframes with these features only - df_lag_train, df_lag_valid, and df_lag_test. The entire procedure is the same as that of Randon Forest Regressor. Only thing is that we need to train the Gradient Boosting Regressor instead of Random Forest Regressor.
#%%
from sklearn.ensemble import GradientBoostingRegressor
#%%
def retrieve_gradient_boosting_model(df, target_variable):
    X = df.drop(columns=[target_variable])
    y = df[target_variable]
    gradient_boosting_regr = GradientBoostingRegressor()
    gradient_boosting_model = gradient_boosting_regr.fit(X,y)
    return gradient_boosting_model
#%%
(
    mae_df,
    rmse_df,
    mae_horizon_error_profile,
    rmse_horizon_error_profile
) = multi_horizon_ensemble_error_profile(
    df_lag,
    validation_window_1,
    "HHS_Care",
```

```
    "gradient boosting",
    size_of_window=7
)
#%%
mae_horizon_error_profile
#%%
rmse_horizon_error_profile
#%% md
Now, we are done with predicting HHS Care, thereby solving the first question - "How many children will be under HHS care in the coming days or weeks ?"
#%% md
Now, we need to consider the second question - " Will discharge capacity be sufficient to offset incoming transfers?" This means that the number of beds vacated by discharged children should be more than or equal to the number of children entering HHS Care from CBP Custody. So here, we need to predict two variables - Number of Children who transferred from CBP to HHS, and the number of children who got discharged from HHS on the previous day.
#%%
df_7.head()
#%%

```